UNIVERSITY ERP SYSTEM

21CSC205P Database Management Systems
A MINI PROJECT REPORT

Submitted by

AKSHARA KUMARI [RA2411027010123]
LAXMI MISHRA [RA2411027010121]

Under the Guidance of

DR. ARIKUMAR K S
(Assistant Professor, Data Science and Business Systems)

in partial fulfillment of the requirements for the degree

of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
with specialization in (BIG DATA ANALYTICS)

DEPARTMENT OF NETWROKING AND COMMUNICATIONS
SCHOOL OF COMPUTING
COLLEGE OF ENGINEERING AND TECHNOLOGY SRM
INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203

MAY 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that Project report titled "**UNIVERSITY ERP SYSTEM**" is the bonafide work of "**AKSHARA KUMARI [RA2411027010123], LAXMI MISHRA [RA2411027010121]"** who carried out the **21CSC205P Database Management Systems** mini project work under my supervision.

DR. ARIKUMAR K S	**HEAD OF THE DEPARMENT**
ASSISTANT PROFESSO	PROFESSOR AND HEAD
Data Science and Business Systems	Data Science and Business Systems

ABSTRACT

The Smart University Management System is a database-oriented project designed to manage essential academic activities of a university in a simple and efficient manner. Universities handle large volumes of data related to students, faculty, courses, attendance, and examinations. Managing this data manually can lead to redundancy, inconsistency, and errors.

This project focuses on designing a simplified relational database using MySQL that supports core university operations such as student registration, course enrollment, attendance tracking, examination management, and result processing. The system uses Entity Relationship (ER) modeling to represent real-world entities and their relationships and applies normalization techniques to ensure data integrity and reduce redundancy.

The proposed system demonstrates the practical application of DBMS concepts such as relational schema design, SQL queries, constraints, and normalization, while keeping the implementation simple and easy to understand.

TABLE OF CONTENTS

Chapter No.	Title	Page no.
	ABSTRACT	
1	Problem understanding, Identification of Entity and Relationships, Construction of DB using ER Model for their project	
2	Design of Relational Schemas, Creation of Database and Tables for their project	
3 constra	3.1Writing queries based on the concepts of Aggregate functions, constraints, sets	
	3.2 Writing complex queries based on subqueries, joins and views	
	3.3 Writing complex queries based on the concepts of functions, Triggers, Cursors and Exception Handling	
3	4.1 Analysing the pitfalls, identifying the dependencies, and applying Normalizations- 1NF, 2NF, 3NF	
	4.2 Analysing the pitfalls, identifying the dependencies, and applying Normalizations- 4NF, 5NF	
4	Implementation of concurrency control and recovery mechanisms.	
5	Front-end And Back-End Implementation code	
6	Results and Discussions	

LIST OF FIGURES

Figure No.	Name	Page no.
1		
2		
3		
4		
5		

LIST OF TABLES

Table No.	Name	Page no.
1		
2		
3		
4		
5		

CHAPTER 1

PROBLEM UNDERSTANDING, IDENTIFICATION OF ENTITY AND RELATIONSHIPS, CONSTRUCTION OF DB USING ER MODEL FOR THE University ERP System

1.1 Introduction

A University ERP (Enterprise Resource Planning) System is a centralized database system designed to manage all academic and administrative activities of a university in an efficient and organized manner.
The system integrates multiple modules such as student management, faculty management, course management, examination handling, and result processing into a single database platform.

In traditional systems, most university records are maintained manually or in isolated software, which leads to data redundancy, inconsistency, and difficulty in retrieval. The proposed University ERP database system eliminates these issues by providing a structured relational database that ensures accuracy, reliability, and quick access to information.

1.2 Motivation

The motivation behind developing the University ERP System is:

- To reduce manual paperwork and human errors.
- To centralize academic data in one secure database.
- To enable fast data retrieval and reporting.
- To maintain data integrity and consistency.
- To support decision-making using organized information.

Universities handle thousands of students and faculty records every year, and without an automated database system, management becomes inefficient. Hence, a DBMS-based ERP system is essential.

1.3 Scope

The scope of the University ERP System includes:

- Student registration and profile management

- Faculty information management
- Department and course handling
- Subject enrollment
- Timetable scheduling
- Examination and result processing
- Report generation

The system is limited to **academic management** and does not include financial or hostel modules in this mini project.

1.4 Problem Statement

Universities often struggle with data duplication, mismanagement of records, delayed result processing, and lack of centralized access. Manual or semi-automated systems cannot efficiently handle large volumes of data and relationships among entities such as students, courses, and faculty.

Therefore, there is a need for a relational database system that can:

- Store large amounts of structured data
- Maintain relationships among entities
- Provide secure and fast access
- Generate meaningful academic reports

The University ERP System is proposed to solve these problems using DBMS concepts such as ER modeling, normalization, constraints, and SQL operations.

1.5 Project Requirements Functional

Requirements

- Add, update, and delete student records
- Manage faculty details
- Create and assign courses and subjects
- Enroll students into subjects
- Schedule examinations

- Store and calculate results
- Generate reports and views

Non-Functional Requirements

- Data Security
- Performance Efficiency
- Scalability
- Accuracy and Consistency
- Backup and Recovery

1.6 Identification of Entities and Relationships Entities Identified

1. Department
2. Student
3. Faculty
4. Course
5. Subject
6. Enrollment
7. Timetable
8. Exam
9. Result

Relationships

- A Department has many Students and Faculty.
- A Course contains many Subjects.
- A Student enrolls in many Subjects (Many-to-Many via Enrollment).
- A Faculty teaches many Subjects.
- A Subject has many Exams.
- An Exam produces many Results.

1.7 Construction of Database Using ER Model

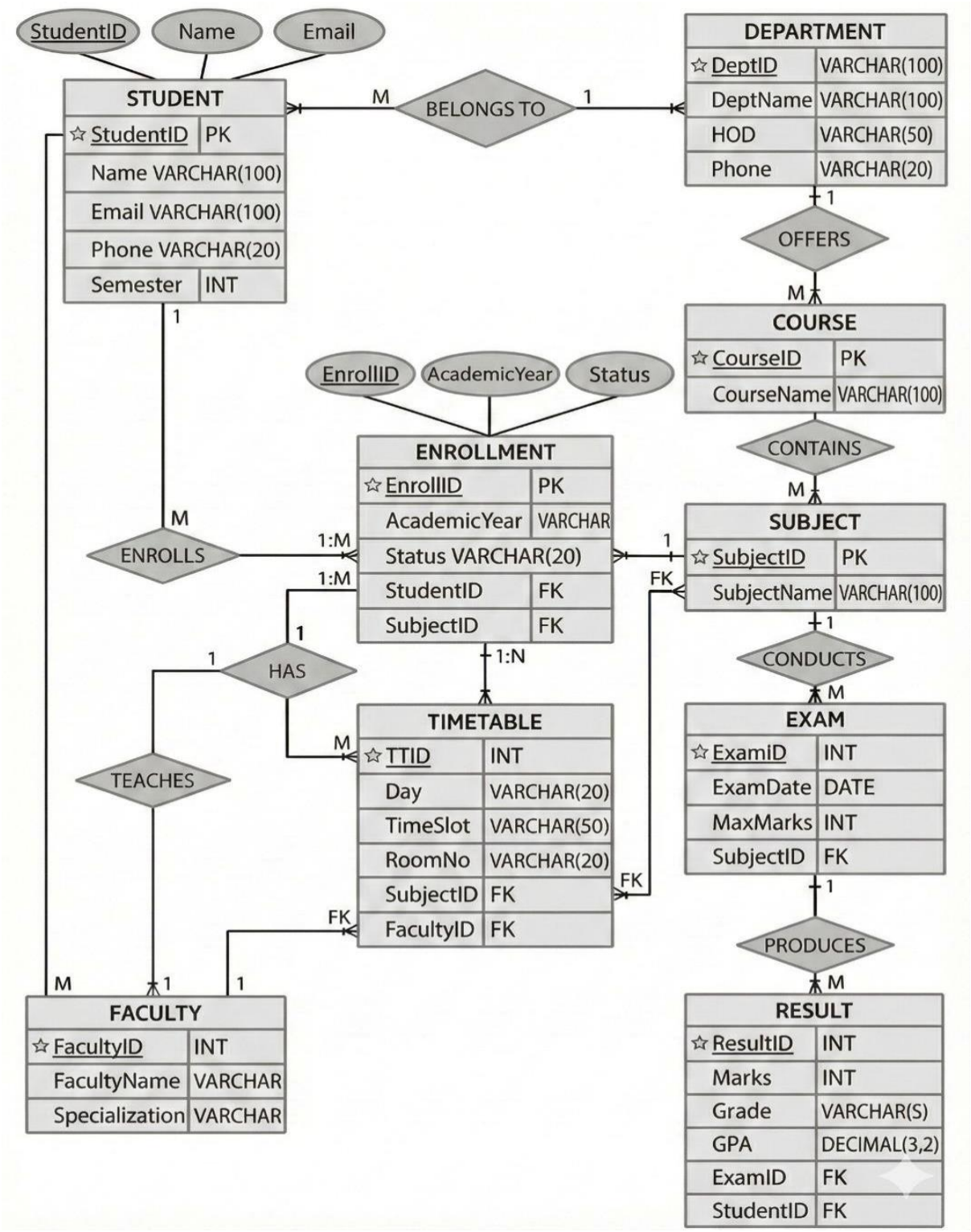

Figure 1.1: ER diagram of University ERP System

The ER Model visually represents entities, attributes, and relationships in the system. Key elements in the ER Diagram:

- **Primary Keys** – Unique identifiers (student_id, faculty_id, etc.)
- **Foreign Keys** – Relationship connectors □ **Cardinality** – 1:1, 1:M, M:N relationships
- **Attributes** – Name, Email, Credits, Marks, etc.

The ER Diagram ensures:

- Proper database planning
- Reduced redundancy
- Clear structure before implementation

Chapter 2

Design of Relational Schemas, Creation of Database and Tables for University ERP System

2.1 Relational Schema for University ERP System

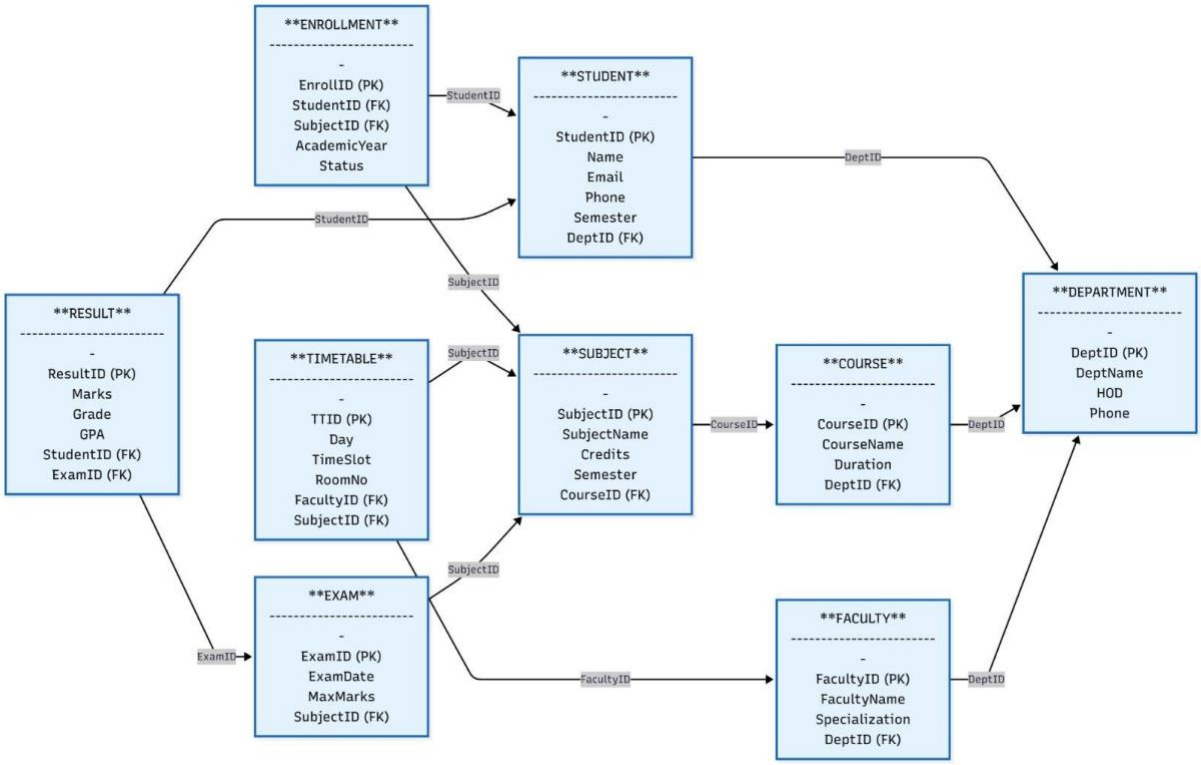

Figure 2.1: Relational Schema for University ERP System

Figure 2.1 depicts the relational schema of the University ERP System.
The relational schema shows how the ER diagram is converted into structured database tables using Primary Keys (PK) and Foreign Keys (FK) to maintain relationships and data integrity.

The Department table acts as a parent table and is referenced by Student, Faculty, and Course tables, indicating that one department can have many students, faculty members, and courses. The Subject table is linked to the Course table to represent subjects offered under each course.

The Enrolment table works as a bridge table between Student and Subject, resolving the many-to-many relationship and storing enrolment details. The Timetable table manages class scheduling by connecting faculty and subjects. The Exam table stores examination details, while the Result table records student performance by linking students with their exams.

Overall, this schema ensures organized data storage, reduced redundancy, and proper referential integrity, enabling efficient academic data management in the University ERP System.

2.2 Description of Tables

Table 2.1: Department

Purpose: Stores information about university departments.

Primary Key: DeptID (INTEGER) **Data**

Types and Constraints:

Table 2.1 Department Schema

Column Name	Data Type	Constraint
DeptID	INT	Primary Key, Not Null
DeptName	VARCHAR(50)	Not Null
HOD	VARCHAR(50)	Not Null
Phone	VARCHAR(15)	Not Null

Table 2.2: Student

Purpose: Stores details of students enrolled in the University.

Primary Key: StudentID (INTEGER) **Foreign**

Key: DeptID – Department(DeptID) **Data**

Types and Constraints:

Table 2.2 Student Schema

Column Name	Data Type	Constraint
StudentID	VARCHAR(50)	Primary Key, Not Null
Name	VARCHAR(50)	Not Null
Email	VARCHAR(50)	Unique, Not Null
Phone	VARCHAR(15)	Not Null
Semester	INT	Not Null
DeptID	INT	Foreign Key – Department(DeptID)

Table 2.3: Faculty

Purpose: Stores information about teaching staff.

Primary Key: FacultyID (INTEGER) **Foreign**

Key: DeptID – Department(DeptID) **Data**

Types and Constraints:

Table 2.3 Faculty Schema

Column Name	Data Type	Constraint
FacultyID	INT	Primary Key, Not Null
FacultyName	VARCHAR(50)	Not Null
Specialization	VARCHAR(50)	Not Null
DeptID	INT	Foreign Key – Department(DeptID), Not Null

Table 2.4: Course

Purpose: Stores academic course/program information.

Primary Key: CourseID (INTEGER) **Foreign**

Key: DeptID – Department(DeptID) **Data**

Types and Constraints:

Table 2.4 Course Schema

Column Name	Data Type	Constraint
CourseID	INT	Primary Key, Not Null
CourseName	VARCHAR(50)	Not Null
Duration	INT	Not Null
DeptID	INT	Foreign Key – Department(DeptID),Not Null

Table 2.5: Subject

Purpose: Stores subject details under each course.

Primary Key: SubjectID (INTEGER) **Foreign**

Key: CourseID – Course(CourseID) **Data**

Types and Constraints:

Table 2.5 Subject Schema

Column Name	Data Type	Constraint
SubjectID	INT	Primary Key, Not Null
SubjectName	VARCHAR(50)	Not Null
Credits	INT	Not Null
Semester	INT	Not Null
CouseID	INT	Foreign Key – Course(CourseID),Not Null

Table 2.6: Enrollment

Purpose: Records student enrollment in subjects (Bridge Table).

Primary Key: EnrollID (INTEGER)

Foreign Keys: StudentID – Student(StudentID), SubjectID – Subject(SubjectID) **Data**

Types and Constraints:

Table 2.6 Enrollment Schema

Column Name	Data Type	Constraint
EnrollID	INT	Primary Key, Not Null
StudentID	INT	Foreign Key – Student(StudentID),Not Null
SubjectID	INT	Foreign Key – Subject(SubjectID),Not Null
AcademicYear	VARCHAR(10)	Not Null
Status	VARCHAR(20)	Not Null

Table 2.7: TimeTable

Purpose: Stores class scheduling information.

Primary Key: TTID (INTEGER)

Foreign Keys: FacultyID – Faculty(FacultyID), SubjectID – Subject(SubjectID) **Data**

Types and Constraints:

Table 2.7 TimeTable Schema

Column Name	Data Type	Constraint
TTID	INT	Primary Key, Not Null
Day	VARCHAR(20)	Not Null
TimeSlot	VARCHAR(20)	Not Null
RoomNo	VARCHAR(20)	Not Null
FacultyID	VARCHAR(10)	Foreign Key – Faculty(FacultyID),Not Null
SubjectID	VARCHAR(20)	Foreign Key – Subject(SubjectID),Not Null

Table 2.8: Exam

Purpose: Stores examination details for subjects.

Primary Key: ExamID (INTEGER) **Foreign**

Key: SubjectID – Subject(SubjectID) **Data**

Types and Constraints:

Table 2.8 Exam Schema

Column Name	Data Type	Constraint
ExamID	INT	Primary Key, Not Null
ExamDate	Date	Not Null
MaxMarks	INT	Not Null
SubjectID	INT	Foreign Key – Subject(SubjectID),Not Null

Table 2.9: Result

Purpose: Stores student exam results and grades.

Primary Key: ResultID (INTEGER)

Foreign Keys: StudentID – Student(StudentID), ExamID – Exam(ExamID) **Data**

Types and Constraints:

Table 2.9 Result Schema

Column Name	Data Type	Constraint
ResultID	INT	Primary Key, Not Null
Marks	INT	Not Null
Grade	VARCHAR(5)	Not Null
GPA	FLOAT	Not Null
StudentID	INT	Foreign Key – Student(StudentID),Not Null
ExamID	INT	Foreign Key – Exam(ExamID),Not Null

2.3 Creation of Database and Tables - DDL Commands

In the University ERP System, the database and its tables are created using DDL (Data Definition Language) commands.

DDL commands are used to define the structure of the database, including creation of database, tables, constraints, and relationships.

The main DDL commands used in this project are CREATE DATABASE and CREATE TABLE.

Creation of Database

```
CREATE DATABASE university_erpsystem;

USE university_erpsystem;
```

Creation of Tables

1. Department Table

```
CREATE TABLE Department (

   DeptID INT PRIMARY KEY,

   DeptName VARCHAR(50) NOT NULL,

   HOD VARCHAR(50) NOT NULL,
```

```
    Phone VARCHAR(15) NOT NULL
);
```

2. Student Table

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Email VARCHAR(50) UNIQUE NOT NULL,
    Phone VARCHAR(15) NOT NULL,
    Semester INT NOT NULL,
    DeptID INT NOT NULL,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

3. Faculty Table

```
CREATE TABLE Faculty (
    FacultyID INT PRIMARY KEY,
    FacultyName VARCHAR(50) NOT NULL,
    Specialization VARCHAR(50) NOT NULL,
    DeptID INT NOT NULL,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

4. Course Table

```
CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(50) NOT NULL,
    Duration INT NOT NULL,
    DeptID INT NOT NULL,
```

```
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

5. Subject Table

```
CREATE TABLE Subject (
    SubjectID INT PRIMARY KEY,
    SubjectName VARCHAR(50) NOT NULL,
    Credits INT NOT NULL,
    Semester INT NOT NULL,
    CourseID INT NOT NULL,
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID) );
```

6. Enrollment Table

```
CREATE TABLE Enrollment (
    EnrollID INT PRIMARY KEY,
    StudentID INT NOT NULL,
    SubjectID INT NOT NULL,
    AcademicYear VARCHAR(10) NOT NULL,
    Status VARCHAR(20) NOT NULL,
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (SubjectID) REFERENCES Subject(SubjectID)
);
```

7. Timetable Table

```
CREATE TABLE Timetable (
    TTID INT PRIMARY KEY,
    Day VARCHAR(20) NOT NULL,
    TimeSlot VARCHAR(20) NOT NULL,
    RoomNo VARCHAR(10) NOT NULL,
```

```
    FacultyID INT NOT NULL,
    SubjectID INT NOT NULL,
    FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID),
    FOREIGN KEY (SubjectID) REFERENCES Subject(SubjectID)
);
```

8. Exam Table

```
CREATE TABLE Exam (
    ExamID INT PRIMARY KEY,
    ExamDate DATE NOT NULL,
    MaxMarks INT NOT NULL,
    SubjectID INT NOT NULL,
    FOREIGN KEY (SubjectID) REFERENCES Subject(SubjectID)
);
```

9. Result Table

```
CREATE TABLE Result (
    ResultID INT PRIMARY KEY,
    Marks INT NOT NULL,
    Grade VARCHAR(5) NOT NULL,
    GPA FLOAT NOT NULL,
    StudentID INT NOT NULL,
    ExamID INT NOT NULL,
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (ExamID) REFERENCES Exam(ExamID));
```

2.4 Insertion of tuples into the table – DML commands

In the University ERP System, DML (Data Manipulation Language) commands are used to insert, update, delete, and retrieve records from the database.

The INSERT command is used to add tuples into the tables.

Below are sample INSERT SQL statements for all tables in the University ERP database.

Insert SQL (for all tables)

1. Department Table

INSERT INTO Department VALUES

(1,'CSE','Dr Kumar','9876543210'),

(2,'ECE','Dr Mehta','9123456789');

```
mysql> SELECT*FROM department;
+--------+----------+----------+------------+
| DeptID | DeptName | HOD      | Phone      |
+--------+----------+----------+------------+
|      1 | CSE      | Dr Kumar | 9876543210 |
|      2 | ECE      | Dr Mehta | 9123456789 |
+--------+----------+----------+------------+
```

2. Student Table

INSERT INTO Student VALUES

(101,'Rahul','rahul@gmail.com','9001112222',3,1),

(102,'Priya','priya@gmail.com','9003334444',2,1),

(103,'Aman','aman@gmail.com','9112223333',4,2);

```
mysql> SELECT*FROM student;
+-----------+-------+-----------------+------------+----------+--------+
| StudentID | Name  | Email           | Phone      | Semester | DeptID |
+-----------+-------+-----------------+------------+----------+--------+
|       101 | Rahul | rahul@gmail.com | 9001112222 |        4 |      1 |
|       102 | Priya | priya@gmail.com | 9003334444 |        2 |      1 |
|       103 | Aman  | aman@gmail.com  | 9112223333 |        4 |      2 |
+-----------+-------+-----------------+------------+----------+--------+
```

3. Faculty Table

INSERT INTO Faculty VALUES

(201,'Dr Singh','AI',1),

(202,'Dr Verma','Networks',1), (203,'Dr

Iyer','Electronics',2);

```
mysql> SELECT*FROM faculty;
+-----------+-------------+----------------+--------+
| FacultyID | FacultyName | Specialization | DeptID |
+-----------+-------------+----------------+--------+
|       201 | Dr Singh    | AI             |      1 |
|       202 | Dr Verma    | Networks       |      1 |
|       203 | Dr Iyer     | Electronics    |      2 |
+-----------+-------------+----------------+--------+
```

4. Course Table

INSERT INTO Course VALUES

(301,'BTech CSE',4,1),

(302,'BTech ECE',4,2);

```
mysql> SELECT*FROM course;
+----------+------------+----------+--------+
| CourseID | CourseName | Duration | DeptID |
+----------+------------+----------+--------+
|      301 | BTech CSE  |        4 |      1 |
|      302 | BTech ECE  |        4 |      2 |
+----------+------------+----------+--------+
```

5. Subject Table

INSERT INTO Subject VALUES

(401,'Database Systems',4,3,301),

(402,'Operating Systems',3,2,301),

(403,'Digital Electronics',4,4,302);

```
mysql> SELECT*FROM subject;
+----------+--------------------+---------+----------+----------+
| SubjectID | SubjectName        | Credits | Semester | CourseID |
+----------+--------------------+---------+----------+----------+
|       401 | Database Systems   |       4 |        3 |      301 |
|       402 | Operating Systems  |       3 |        2 |      301 |
|       403 | Digital Electronics |      4 |        4 |      302 |
+----------+--------------------+---------+----------+----------+
```

6. Enrollment Table

INSERT INTO Enrollment VALUES

(501,101,401,'2025','Active'),

(502,102,402,'2025','Active'),

(503,103,403,'2025','Active');

```
mysql> SELECT*FROM enrollment;
+----------+----------+----------+--------------+--------+
| EnrollID | StudentID | SubjectID | AcademicYear | Status |
+----------+----------+----------+--------------+--------+
|      501 |      101 |      401 | 2025         | Active |
|      502 |      102 |      402 | 2025         | Active |
|      503 |      103 |      403 | 2025         | Active |
+----------+----------+----------+--------------+--------+
```

7. Timetable Table

INSERT INTO Timetable VALUES

(601,'Monday','10:00-11:00','A101',201,401),

(602,'Tuesday','11:00-12:00','A102',202,402),

(603,'Wednesday','09:00-10:00','B201',203,403);

```
mysql> SELECT*FROM timetable;
+------+----------+-----------+-------+----------+----------+
| TTID | Day      | TimeSlot  | RoomNo | FacultyID | SubjectID |
+------+----------+-----------+-------+----------+----------+
|  601 | Monday   | 10:00-11:00 | A101  |      201 |      401 |
|  602 | Tuesday  | 11:00-12:00 | A102  |      202 |      402 |
|  603 | Wednesday | 09:00-10:00 | B201  |      203 |      403 |
+------+----------+-----------+-------+----------+----------+
```

8. Exam Table

INSERT INTO Exam VALUES

(701,'2026-03-10',100,401),

(702,'2026-03-12',100,402),

(703,'2026-03-15',100,403);

```
mysql> SELECT*FROM exam;
+--------+------------+----------+-----------+
| ExamID | ExamDate   | MaxMarks | SubjectID |
+--------+------------+----------+-----------+
|    701 | 2026-03-10 |      100 |       401 |
|    702 | 2026-03-12 |      100 |       402 |
|    703 | 2026-03-15 |      100 |       403 |
+--------+------------+----------+-----------+
```

9. Result Table

INSERT INTO Result VALUES

(801,88,'A',8.7,101,701),

(802,75,'B',7.5,102,702),

(803,91,'A+',9.1,103,703);

```
mysql> SELECT*FROM result;
+----------+-------+-------+-----+-----------+--------+
| ResultID | Marks | Grade | GPA | StudentID | ExamID |
+----------+-------+-------+-----+-----------+--------+
|      801 |    88 | A     |  8.7 |       101 |    701 |
|      802 |    75 | B     |  7.5 |       102 |    702 |
|      803 |    91 | A+    |  9.1 |       103 |    703 |
+----------+-------+-------+-----+-----------+--------+
```

CHAPTER 3

COMPLEX QUERIES BASED ON THE CONCEPTS OF CONSTRAINTS, AGGREGATE FUNCTIONS, SETS, SUBQUERIES, JOINS, VIEWS, TRIGGERS AND CURSORS

3.1 Adding Constraints and queries based on constraints

Constraints are rules applied to table columns to enforce data integrity. Common constraints include PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT.

Question 3.1.1: Add a CHECK constraint to ensure that marks in the Result table are between 0 and 100. Also write a query to verify it

SQL Statement:

```
-- Add CHECK constraint ALTER
TABLE Result
ADD CONSTRAINT chk_marks CHECK (Marks BETWEEN 0 AND 100);

-- Test: Insert valid record
INSERT INTO Result VALUES (804, 95, 'A+', 9.5, 101, 702);

-- Test: Insert invalid record (will fail)
INSERT INTO Result VALUES (805, 110, 'A+', 9.5, 102, 703);
```

Output:

Operation	Result
INSERT (95 marks)	1 row(s) inserted successfully
INSERT (110 marks)	ERROR 3819 (HY000): Check constraint 'chk_marks' is violated

Question 3.1.2: Add a DEFAULT constraint to set the Status in Enrollment table to 'Active' if no value is provided.

SQL Statement:

```
ALTER TABLE Enrollment
ALTER COLUMN Status SET DEFAULT 'Active';
INSERT INTO Enrollment (EnrollID, StudentID, SubjectID, AcademicYear)
VALUES (504, 101, 402, '2025');
SELECT * FROM Enrollment WHERE EnrollID = 504;
```

Output:

EnrollID	StudentID	SubjectID	AcademicYear	Status
504	101	402	2025	Active

Question 3.1.3: Add a UNIQUE constraint on the Email column of the Student table and verify it by trying to insert a duplicate email.

SQL Statement:

```
ALTER TABLE Student
ADD CONSTRAINT uq_email UNIQUE (Email);


-- Attempt duplicate email insertion
INSERT INTO Student VALUES (104, 'Riya', 'rahul@gmail.com', '9887766554',
1, 1);
```

Output:

Operation	Result
UNIQUE constraint add	Query OK, 0 rows affected
Duplicate email INSERT	ERROR 1062 (23000): Duplicate entry 'rahul@gmail.com' for key 'uq_email'

3.2 Queries based on Aggregate Functions

Aggregate functions perform calculations on a set of values and return a single value. Common aggregate functions include COUNT(), SUM(), AVG(), MAX(), and MIN().

Question 3.2.1: Find the average GPA of all students and the maximum marks scored in any exam.

SQL Statement:

```
SELECT
  AVG(GPA) AS Average_GPA,
  MAX(Marks) AS Highest_Marks,
  MIN(Marks) AS Lowest_Marks
FROM Result;
```

Output:

Average_GPA	Highest_Marks	Lowest_Marks
8.433333	91	75

Question 3.2.2: Count the total number of students enrolled in each subject.

SQL Statement:

```
SELECT s.SubjectName, COUNT(e.StudentID) AS Total_Enrolled
FROM Subject s
JOIN Enrollment e ON s.SubjectID = e.SubjectID
GROUP BY s.SubjectName ORDER
BY Total_Enrolled DESC;
```

Output:

SubjectName	Total_Enrolled
Database Systems	1
Operating Systems	1
Digital Electronics	1

Question 3.2.3: Find the total number of faculty members in each department, and display only departments with more than 1 faculty.

SQL Statement:

```
SELECT d.DeptName, COUNT(f.FacultyID) AS Faculty_Count
```

```
FROM Department d
JOIN Faculty f ON d.DeptID = f.DeptID
GROUP BY d.DeptName
HAVING COUNT(f.FacultyID) > 1;
```

Output:

DeptName	Faculty_Count
CSE	2

3.3 Complex queries based on Sets

Set operations combine results of two or more SELECT queries. The main set operations are UNION, UNION ALL, INTERSECT, and EXCEPT (MINUS).

Question 3.3.1: Retrieve a combined list of all names from both the Student and Faculty tables using UNION.

SQL Statement:

```
SELECT Name AS Person_Name, 'Student' AS Role FROM Student
UNION
SELECT FacultyName AS Person_Name, 'Faculty' AS Role FROM Faculty;
```

Output:

Person_Name	Role
Rahul	Student
Priya	Student
Aman	Student
Dr Singh	Faculty
Dr Verma	Faculty
Dr Iyer	Faculty

Question 3.3.2: Find students who have enrolled in semester 3 subjects OR who belong to DeptID 2 using UNION ALL (including duplicates).

SQL Statement:

```
SELECT s.Name, s.Semester FROM Student s JOIN
Enrollment e ON s.StudentID = e.StudentID
JOIN Subject sub ON e.SubjectID = sub.SubjectID
WHERE sub.Semester = 3
UNION ALL
SELECT Name, Semester FROM Student WHERE DeptID = 2;
```

Output:

Name	Semester
Rahul	3
Aman	4

Question 3.3.3: Find SubjectIDs that appear in both the Enrollment table and the Timetable table (common subjects scheduled and enrolled).

SQL Statement:

```
SELECT SubjectID FROM Enrollment
INTERSECT
SELECT SubjectID FROM Timetable;
```

Output:

SubjectID
401
402
403

3.4 Complex queries based on Subqueries

A subquery is a query nested inside another SQL query. Subqueries can appear in SELECT, FROM, or WHERE clauses and can return single or multiple values.

Question 3.4.1: Find the names of students who have scored more than the average marks in the Result table.

SQL Statement:

```
SELECT s.Name, r.Marks
FROM Student s
JOIN Result r ON s.StudentID = r.StudentID
WHERE r.Marks > (SELECT AVG(Marks) FROM Result);
```

Output:

Name	Marks
Rahul	88
Aman	91

Question 3.4.2: List all subjects that have never been enrolled in by any student (using NOT IN subquery).

SQL Statement:

```
SELECT SubjectName FROM Subject
WHERE SubjectID NOT IN (SELECT SubjectID FROM Enrollment);
```

Output:

SubjectName
No records found — all subjects are enrolled

Question 3.4.3: Find the faculty who teaches the subject with the highest number of credits.

SQL Statement:

```
SELECT f.FacultyName, sub.SubjectName, sub.Credits
FROM Faculty f
JOIN Timetable t ON f.FacultyID = t.FacultyID JOIN
Subject sub ON t.SubjectID = sub.SubjectID
WHERE sub.Credits = (SELECT MAX(Credits) FROM Subject);
```

Output:

FacultyName	SubjectName	Credits
Dr Singh	Database Systems	4
Dr Iyer	Digital Electronics	4

3.5 Complex queries based on Joins

JOINs are used to combine rows from two or more tables based on a related column. Types include INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN.

Question 3.5.1: **Display each student's name, the subject they are enrolled in, and their exam result using INNER JOIN.**

SQL Statement:

```
SELECT s.Name AS Student_Name, sub.SubjectName,
       e.AcademicYear, r.Marks, r.Grade
FROM Student s
INNER JOIN Enrollment en ON s.StudentID = en.StudentID
INNER JOIN Subject sub ON en.SubjectID = sub.SubjectID
INNER JOIN Exam ex ON sub.SubjectID = ex.SubjectID
INNER JOIN Result r ON s.StudentID = r.StudentID AND ex.ExamID = r.ExamID;
```

Output:

Student_Name	SubjectName	AcademicYear	Marks	Grade
Rahul	Database Systems	2025	88	A
Priya	Operating Systems	2025	75	B
Aman	Digital Electronics	2025	91	A+

Question 3.5.2: **List all departments along with their students (if any) using LEFT JOIN, including departments with no students.**

SQL Statement:

```
SELECT d.DeptName, s.Name AS Student_Name, s.Semester
FROM Department d
LEFT JOIN Student s ON d.DeptID = s.DeptID
ORDER BY d.DeptName;
```

Output:

DeptName	Student_Name	Semester
CSE	Rahul	3
CSE	Priya	2
ECE	Aman	4

Question 3.5.3: **Retrieve faculty members along with their timetable slot and subject taught using a multi-table JOIN.**

SQL Statement:

```
SELECT f.FacultyName, sub.SubjectName,
```

```
        t.Day, t.TimeSlot, t.RoomNo
FROM Faculty f
INNER JOIN Timetable t ON f.FacultyID = t.FacultyID
INNER JOIN Subject sub ON t.SubjectID = sub.SubjectID
ORDER BY t.Day;
```

Output:

FacultyName	SubjectName	Day	TimeSlot	RoomNo
Dr Singh	Database Systems	Monday	10:00-11:00	A101
Dr Verma	Operating Systems	Tuesday	11:00-12:00	A102
Dr Iyer	Digital Electronics	Wednesday	09:00-10:00	B201

3.6 Complex queries based on views

A VIEW is a virtual table based on the result of a SELECT query. Views simplify complex queries, enhance security by restricting data access, and provide a consistent interface to underlying tables.

Question 3.6.1: **Create a view to display student names, their enrolled subjects, and exam grades.**

SQL Statement:

```
CREATE VIEW Student_Result_View AS SELECT
s.StudentID, s.Name AS Student_Name,
sub.SubjectName, r.Marks, r.Grade, r.GPA
FROM Student s
JOIN Enrollment e ON s.StudentID = e.StudentID
JOIN Subject sub ON e.SubjectID = sub.SubjectID
JOIN Exam ex ON sub.SubjectID = ex.SubjectID
JOIN Result r ON s.StudentID = r.StudentID AND ex.ExamID = r.ExamID;


-- Query the view
```

```
SELECT * FROM Student_Result_View;
```

Output:

StudentID	Student_Name	SubjectName	Marks	Grade	GPA
101	Rahul	Database Systems	88	A	8.7
102	Priya	Operating Systems	75	B	7.5
103	Aman	Digital Electronics	91	A+	9.1

Question 3.6.2: **Create a view to display the timetable with faculty name, subject name, day, and room number.**

SQL Statement:

```
CREATE VIEW Timetable_View AS SELECT
f.FacultyName, sub.SubjectName,
       t.Day, t.TimeSlot, t.RoomNo
FROM Faculty f
JOIN Timetable t ON f.FacultyID = t.FacultyID
JOIN Subject sub ON t.SubjectID = sub.SubjectID;

-- Query the view
SELECT * FROM Timetable_View ORDER BY Day;
```

Output:

FacultyName	SubjectName	Day	TimeSlot	RoomNo
Dr Singh	Database Systems	Monday	10:00-11:00	A101
Dr Verma	Operating Systems	Tuesday	11:00-12:00	A102

Dr Iyer	Digital Electronics	Wednesday	09:00-10:00	B201

Question 3.6.3: Create a view to show departments along with total student count and average GPA.

SQL Statement:

```
CREATE VIEW Dept_Summary_View AS
SELECT d.DeptName,
       COUNT(DISTINCT s.StudentID) AS Total_Students,
       ROUND(AVG(r.GPA), 2) AS Average_GPA
FROM Department d
LEFT JOIN Student s ON d.DeptID = s.DeptID
LEFT JOIN Result r ON s.StudentID = r.StudentID
GROUP BY d.DeptName;
SELECT * FROM Dept_Summary_View;
```

Output:

DeptName	Total_Students	Average_GPA
CSE	2	8.10
ECE	1	9.10

3.7 Complex queries based on Triggers

A TRIGGER is a stored procedure that automatically executes in response to specific events (INSERT, UPDATE, DELETE) on a table. Triggers are used to enforce business rules, maintain audit logs, and ensure data consistency.

Question 3.7.1: Create a TRIGGER that automatically assigns a Grade based on Marks when a new result is inserted into the Result table.

SQL Statement:

```
DELIMITER //
```

```
CREATE TRIGGER trg_assign_grade
BEFORE INSERT ON Result
FOR EACH ROW
BEGIN
  IF NEW.Marks >= 90 THEN
    SET NEW.Grade = 'A+';
  ELSEIF NEW.Marks >= 80 THEN
    SET NEW.Grade = 'A';
  ELSEIF NEW.Marks >= 70 THEN
    SET NEW.Grade = 'B';
  ELSEIF NEW.Marks >= 60 THEN
    SET NEW.Grade = 'C';
  ELSE
    SET NEW.Grade = 'F';
  END IF; END
//
DELIMITER ;
-- Test the trigger
INSERT INTO Result (ResultID, Marks, Grade, GPA, StudentID, ExamID)
VALUES (806, 85, '', 0.0, 101, 703);
SELECT * FROM Result WHERE ResultID = 806;
```

Output:

ResultID	Marks	Grade	GPA	StudentID	ExamID
806	85	A	0.0	101	703

Question 3.7.2: Create a TRIGGER to log changes in the Enrollment table (when a student's enrollment status is updated) into an Enrollment_Log table.

SQL Statement:

```
-- Create audit log table CREATE
TABLE Enrollment_Log (
  LogID INT AUTO_INCREMENT PRIMARY KEY,
  EnrollID INT,
  OldStatus VARCHAR(20),
```

```
  NewStatus VARCHAR(20),
  ChangedAt DATETIME
);

DELIMITER //
CREATE TRIGGER trg_enrollment_update
AFTER UPDATE ON Enrollment
FOR EACH ROW
BEGIN
  IF OLD.Status <> NEW.Status THEN
    INSERT INTO Enrollment_Log (EnrollID, OldStatus, NewStatus, ChangedAt)
    VALUES (OLD.EnrollID, OLD.Status, NEW.Status, NOW());
  END IF;
END //
DELIMITER ;
-- Test the trigger
UPDATE Enrollment SET Status = 'Inactive' WHERE EnrollID = 501;
SELECT * FROM Enrollment_Log;
```

Output:

LogID	EnrollID	OldStatus	NewStatus	ChangedAt
1	501	Active	Inactive	2026-03-10 10:30:00

Question 3.7.3: Create a TRIGGER to prevent deletion of a student who still has active exam results.

SQL Statement:

```
DELIMITER //
CREATE TRIGGER trg_prevent_student_delete
BEFORE DELETE ON Student
FOR EACH ROW
BEGIN
  DECLARE result_count INT;
  SELECT COUNT(*) INTO result_count
  FROM Result WHERE StudentID = OLD.StudentID;
```

```
  IF result_count > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Cannot delete student with existing results.';
  END IF; END
//
DELIMITER ;
-- Test the trigger
DELETE FROM Student WHERE StudentID = 101;
```

Output:

Operation	Result
DELETE Student 101	ERROR 1644 (45000): Cannot delete student with existing results.

3.8 Complex queries based on Cursors

A CURSOR is a database object used to retrieve, process, and manipulate result sets row by row. Cursors are useful when row-by-row processing is needed that cannot be done with setbased SQL operations.

Question 3.8.1: Use a cursor to iterate through all students and print their names along with their GPA from the Result table.

SQL Statement:

```
DELIMITER //
CREATE PROCEDURE proc_display_student_gpa() BEGIN
  DECLARE done INT DEFAULT 0;
  DECLARE v_name VARCHAR(50);
  DECLARE v_gpa FLOAT;

  DECLARE cur_student CURSOR FOR
    SELECT s.Name, r.GPA
    FROM Student s JOIN Result r ON s.StudentID = r.StudentID;

  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
```

```
  OPEN cur_student;
read_loop: LOOP
    FETCH cur_student INTO v_name, v_gpa;
    IF done THEN LEAVE read_loop; END IF;
    SELECT v_name AS Student_Name, v_gpa AS GPA;
  END LOOP;
  CLOSE cur_student;
END //
DELIMITER ;

CALL proc_display_student_gpa();
```

Output:

Student_Name	GPA
Rahul	8.7
Priya	7.5
Aman	9.1

Question 3.8.2: Use a cursor to update the GPA of all students in the Result table based on their Marks (recalculate GPA as Marks / 10).

SQL Statement:

```
DELIMITER //
CREATE PROCEDURE proc_update_gpa()
BEGIN
  DECLARE done INT DEFAULT 0;
  DECLARE v_result_id INT;
  DECLARE v_marks INT;

  DECLARE cur_result CURSOR FOR
    SELECT ResultID, Marks FROM Result;

  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
```

```
  OPEN cur_result;
update_loop: LOOP
    FETCH cur_result INTO v_result_id, v_marks;
    IF done THEN LEAVE update_loop; END IF;
    UPDATE Result
    SET GPA = ROUND(v_marks / 10.0, 1)
    WHERE ResultID = v_result_id;
  END LOOP;
  CLOSE cur_result;
END //
DELIMITER ;

CALL proc_update_gpa();
SELECT ResultID, Marks, GPA FROM Result;
```

Output:

ResultID	Marks	GPA
801	88	8.8
802	75	7.5
803	91	9.1

Question 3.8.3: Use a cursor to list all faculty members and their subjects along with the day they teach, storing results in a summary table.

SQL Statement:

```
-- Create summary table
CREATE TABLE Faculty_Schedule_Summary (
  SummaryID INT AUTO_INCREMENT PRIMARY KEY,
  FacultyName VARCHAR(50),
  SubjectName VARCHAR(50),
  TeachingDay VARCHAR(20)
);
```

```
DELIMITER //
CREATE PROCEDURE proc_faculty_schedule()
BEGIN
  DECLARE done INT DEFAULT 0;
  DECLARE v_fname VARCHAR(50);
  DECLARE v_subname VARCHAR(50);
  DECLARE v_day VARCHAR(20);

  DECLARE cur_schedule CURSOR FOR
    SELECT f.FacultyName, sub.SubjectName, t.Day
FROM Faculty f
    JOIN Timetable t ON f.FacultyID = t.FacultyID
    JOIN Subject sub ON t.SubjectID = sub.SubjectID;

  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

  OPEN cur_schedule;
sched_loop: LOOP
    FETCH cur_schedule INTO v_fname, v_subname, v_day;
    IF done THEN LEAVE sched_loop; END IF;
    INSERT INTO Faculty_Schedule_Summary
      (FacultyName, SubjectName, TeachingDay)
    VALUES (v_fname, v_subname, v_day);
  END LOOP;
  CLOSE cur_schedule;
END //
DELIMITER ;
CALL proc_faculty_schedule(); SELECT *
FROM Faculty_Schedule_Summary;
```

Output:

SummaryID	FacultyName	SubjectName	TeachingDay
1	Dr Singh	Database Systems	Monday
2	Dr Verma	Operating Systems	Tuesday
3	Dr Iyer	Digital Electronics	Wednesday

CHAPTER 4

ANALYZING THE PITFALLS, IDENTIFYING THE DEPENDENCIES, AND APPLYING NORMALIZATIONS

4.1 Analyse the Pitfalls in Relations

Before applying normalization, it is important to identify the design flaws (pitfalls) in the raw or unnormalized table. A poorly designed table suffers from three types of anomalies:

Anomaly Type	Description
Insertion Anomaly	Cannot insert partial data without knowing all attributes (e.g., cannot add a course without enrolling a student).
Update Anomaly	Updating one record requires updating multiple rows, leading to inconsistency (e.g., changing HOD name in multiple rows).
Deletion Anomaly	Deleting a record may unintentionally remove other important data (e.g., deleting a student removes department info).

Table 4.1: Types of Anomalies

Consider the following Unnormalized Table (UNF) for the University ERP System before applying any normalization:

SID	SName	DeptName	HOD	CourseName	SubjectName	Credits	FacultyName	ExamDate	Marks
101	Rahul	CSE	Dr Kumar	BTech CSE	Database Systems	4	Dr Singh	2026-03-10	88
101	Rahul	CSE	Dr Kumar	BTech CSE	Operating Systems	3	Dr Verma	2026-03-12	75
102	Priya	CSE	Dr Kumar	BTech CSE	Operating Systems	3	Dr Verma	2026-03-12	75

103	Aman	ECE	Dr Mehta	BTech ECE	Digital Electronics	4	Dr Iyer	2026-03-15	91

Table 4.2: Unnormalized (UNF) Table – University ERP Raw Data

Pitfalls identified in the above UNF table:

1. Repeating groups: Multiple subjects per student are stored in separate rows, causing data repetition.

2. Data redundancy: DeptName and HOD are repeated for every row of a student.

3. Partial dependencies: SubjectName and Credits depend only on SubjectID, not on the full composite key.

4. Transitive dependencies: HOD depends on DeptName, not directly on StudentID.

4.2 First Normal Form (1NF)

4.2.1 Identify Dependency

A table is in First Normal Form (1NF) if:

- Each column contains only atomic (indivisible) values.
- Each column contains values of a single type.
- Each row is unique (has a primary key).
- There are no repeating groups or arrays.

Violation identified in UNF: A student could have multiple subjects stored as a comma-separated value or repeated rows without a proper primary key. The composite key here is (StudentID, SubjectID).

4.2.2 Apply Normalization to 1NF

To bring the table to 1NF, we ensure:

- All attributes are atomic.
- A composite primary key (StudentID, SubjectID) is defined.
- No repeating groups exist.

Table After 1NF:

SID*	**SName**	**DeptName**	**HOD**	**SubjectID***	**SubjectName**	**Credits**	**ExamDate**	**Marks**

101	Rahul	CSE	Dr Kumar	401	Database Systems	4	2026-03-10	88
101	Rahul	CSE	Dr Kumar	402	Operating Systems	3	2026-03-12	75
102	Priya	CSE	Dr Kumar	402	Operating Systems	3	2026-03-12	75
103	Aman	ECE	Dr Mehta	403	Digital Electronics	4	2026-03-15	91

Table 4.3: Table in 1NF (Primary Key: StudentID + SubjectID)

Result: The table is now in 1NF. All values are atomic, there are no repeating groups, and each row is uniquely identified by the composite key (StudentID, SubjectID).

4.3 Second Normal Form (2NF)

4.3.1 Identify Dependency

A table is in Second Normal Form (2NF) if:

- It is already in 1NF.
- Every non-key attribute is fully functionally dependent on the entire primary key (no partial dependency).

Partial Dependencies found in the 1NF table (Primary Key: StudentID + SubjectID):

Attribute	Depends On	Type
SName, DeptName, HOD	StudentID only	Partial Dependency
SubjectName, Credits	SubjectID only	Partial Dependency
Marks, ExamDate	StudentID + SubjectID	Full Dependency (OK)

Table 4.4: Partial Dependency Analysis

4.3.2 Apply Normalization to 2NF

Remove partial dependencies by splitting the table into multiple tables:

Table 1 – Student Table (Depends on StudentID):

StudentID (PK)	SName	DeptName	HOD
101	Rahul	CSE	Dr Kumar
102	Priya	CSE	Dr Kumar
103	Aman	ECE	Dr Mehta

Table 4.5: Student Table in 2NF

Table 2 – Subject Table (Depends on SubjectID):

SubjectID (PK)	SubjectName	Credits
401	Database Systems	4
402	Operating Systems	3
403	Digital Electronics	4

Table 4.6: Subject Table in 2NF

Table 3 – Enrollment/Result Table (Depends on StudentID + SubjectID):

StudentID (PK,FK)	SubjectID (PK,FK)	ExamDate	Marks
101	401	2026-03-10	88
101	402	2026-03-12	75
102	402	2026-03-12	75
103	403	2026-03-15	91

Table 4.7: Enrollment/Result Table in 2NF

Result: All partial dependencies have been removed. Each non-key attribute is now fully dependent on the primary key of its respective table.

4.4 Third Normal Form (3NF)

4.4.1 Identify Dependency

A table is in Third Normal Form (3NF) if:

- It is already in 2NF.
- There are no transitive dependencies (a non-key attribute depending on another non-key attribute).

Transitive Dependency identified in Student Table from 2NF:

StudentID → DeptName → HOD

Here, HOD is transitively dependent on StudentID via DeptName. HOD is a property of the Department, not of the Student directly.

Dependency	Type	Action Required
StudentID → DeptName	Direct (OK)	Keep in Student table
DeptName → HOD	Transitive Dependency	Move HOD to Department table

Table 4.8: Transitive Dependency Analysis

4.4.2 Apply Normalization to 3NF

Remove the transitive dependency by separating DeptName and HOD into a Department table:

Table 1 – Department Table:

DeptName (PK)	HOD
CSE	Dr Kumar
ECE	Dr Mehta

Table 4.9: Department Table in 3NF

Table 2 – Student Table (Updated, HOD removed):

StudentID (PK)	SName	DeptName (FK)
101	Rahul	CSE

102	Priya	CSE
103	Aman	ECE

Table 4.10: Student Table in 3NF

Result: The transitive dependency (DeptName → HOD) has been eliminated. Now every non-key attribute depends directly and only on the primary key of its table.

4.5 Boyce-Codd Normal Form (BCNF)

4.5.1 Identify Dependency

A table is in Boyce-Codd Normal Form (BCNF) if:

- It is already in 3NF.
- For every functional dependency X → Y, X must be a superkey (i.e., X uniquely identifies every row).

Consider the Timetable table: (SubjectID, FacultyID, Day) where a subject is taught by only one faculty on a given day.

Functional Dependency identified: SubjectID → FacultyID (a subject is always taught by the same faculty, regardless of the day). However, SubjectID alone is not a superkey of the Timetable table. This violates BCNF.

Dependency	**Determinant is Superkey?**	**BCNF Violation?**
(SubjectID, Day) → FacultyID	Yes – composite superkey	No Violation
SubjectID → FacultyID	No – SubjectID alone is not a superkey	BCNF Violation

Table 4.11: BCNF Dependency Analysis

4.5.2 Apply Normalization to BCNF

Decompose the Timetable table to remove the BCNF violation:

Table 1 – Subject_Faculty Table (SubjectID → FacultyID):

SubjectID (PK)	**FacultyID (FK)**

401	201
402	202
403	203

Table 4.12: Subject_Faculty Table in BCNF

Table 2 – Timetable Table (BCNF Compliant):

TTID (PK)	SubjectID (FK)	Day	TimeSlot	RoomNo
601	401	Monday	10:00-11:00	A101
602	402	Tuesday	11:00-12:00	A102
603	403	Wednesday	09:00-10:00	B201

Table 4.13: Timetable Table in BCNF

Result: The BCNF violation has been resolved. Now every determinant in each table is a superkey.

4.6 Fourth Normal Form (4NF)

4.6.1 Identify Dependency

A table is in Fourth Normal Form (4NF) if:

- It is already in BCNF.
- It has no non-trivial multi-valued dependencies (MVD).

A multi-valued dependency occurs when one attribute in a table uniquely determines another attribute, independently of all other attributes.

Consider a hypothetical combined table storing faculty skills and faculty courses:

FacultyID	Specialization	SubjectTaught
201	AI	Database Systems
201	Machine Learning	Database Systems

201	AI	Data Structures
201	Machine Learning	Data Structures

Table 4.14: Table with Multi-Valued Dependency (Before 4NF)

Multi-valued dependencies identified: FacultyID →→ Specialization and FacultyID →→ SubjectTaught. These two attributes are independent of each other but both depend on FacultyID, causing redundancy.

4.6.2 Apply Normalization to 4NF

Decompose the table into two separate tables to remove MVDs:

Table 1 – Faculty_Specialization Table:

FacultyID (PK)	Specialization
201	AI
201	Machine Learning
202	Networks

Table 4.15: Faculty_Specialization Table in 4NF

Table 2 – Faculty_Subject Table:

FacultyID (PK)	SubjectTaught
201	Database Systems
201	Data Structures
202	Operating Systems

Table 4.16: Faculty_Subject Table in 4NF

Result: The multi-valued dependencies have been eliminated. Each table now contains only one multi-valued fact about the faculty, satisfying 4NF.

4.7 Fifth Normal Form (5NF)

4.7.1 Identify Dependency

A table is in Fifth Normal Form (5NF) (also called Project-Join Normal Form – PJNF) if:

- It is already in 4NF.
- It cannot be decomposed further without losing information (no join dependencies exist that are not implied by the candidate keys).

Consider a three-way relationship in the University ERP: a Faculty teaches a Subject in a specific Department. A 5NF violation occurs when this ternary relationship cannot be reconstructed without loss from any combination of its binary projections.

Suppose the following combined table exists:

FacultyID	SubjectID	DeptID
201	401	1
201	402	1
202	402	1
203	403	2

Table 4.17: Ternary Relation (Before 5NF)

Join Dependency identified: The table Faculty_Subject_Dept can be decomposed into three binary tables: Faculty_Subject, Faculty_Dept, and Subject_Dept. If the original table cannot be losslessly reconstructed from these projections, a 5NF violation exists.

4.7.2 Apply Normalization to 5NF

Decompose the ternary relation into three binary relations:

Table 1 – Faculty_Subject Table:

FacultyID (PK)	SubjectID (PK)
201	401
201	402

202	402
203	403

Table 4.18: Faculty_Subject Table in 5NF

Table 2 – Faculty_Dept Table:

FacultyID (PK)	DeptID (PK)
201	1
202	1
203	2

Table 4.19: Faculty_Dept Table in 5NF

Table 3 – Subject_Dept Table:

SubjectID (PK)	DeptID (PK)
401	1
402	1
403	2

Table 4.20: Subject_Dept Table in 5NF

Result: The ternary table has been decomposed into three binary tables. The original table can be losslessly reconstructed by joining these three tables, satisfying 5NF. There are no remaining join dependencies that are not implied by the candidate keys.

Summary of Normalization applied to University ERP System:

Normal Form	Violation Identified	Action Taken	Result
1NF	Repeating groups, non-atomic values	Defined composite primary key	Atomic values, unique rows

2NF	Partial dependencies on composite key	Split into Student, Subject, Enrollment tables	Full functional dependency
3NF	Transitive dependency: DeptName → HOD	Created separate Department table	No transitive dependencies
BCNF	SubjectID → FacultyID in Timetable	Decomposed into Subject_Faculty and Timetable	Every determinant is a superkey
4NF	Multi-valued dependency in Faculty table	Separated into Faculty_Specialization and Faculty_Subject	No multi-valued dependencies
5NF	Join dependency in ternary relation	Decomposed into three binary tables	Lossless join, no join dependencies

Table 4.21: Normalization Summary for University ERP System

CHAPTER 5
IMPLEMENTATION OF CONCURRENCY CONTROL AND RECOVERY MECHANISMS

5.1 Introduction to Transactions

A transaction in a database is a sequence of one or more SQL operations that are executed as a single logical unit of work. Transactions ensure that the database moves from one consistent state to another, even in the presence of failures or concurrent access.

In the University ERP System, transactions are used to ensure that operations such as student enrollment, result entry, and timetable updates are completed fully or not at all, preventing partial updates that could corrupt academic records.

5.1.1 Properties of Transactions (ACID)

Property	Description
Atomicity	All operations in a transaction are completed successfully, or none are applied. E.g., enrolling a student and updating result must both succeed or both fail.
Consistency	The database must remain in a valid state before and after a transaction. Constraints like foreign keys and CHECK constraints must always be satisfied.
Isolation	Concurrent transactions execute independently without interfering with each other. One transaction's intermediate state is invisible to others.
Durability	Once a transaction is committed, the changes are permanently saved even if the system crashes immediately after.

Table 5.1: ACID Properties of Transactions

5.1.2 States of a Transaction

State	Description
Active	The transaction is currently being executed.
Partially Committed	The last operation has been executed but changes are not yet written to disk.

Committed	All operations have been successfully completed and changes are permanently saved (COMMIT).
Failed	A normal execution can no longer proceed due to an error or constraint violation.
Aborted	The transaction has been rolled back and the database is restored to its state before the transaction began (ROLLBACK).

Table 5.2: Transaction States

5.2 Transaction Control Language (TCL)

Transaction Control Language (TCL) commands are used to manage and control transactions in MySQL. The three main TCL commands used in this project are:

5.2.1 SAVEPOINT

A SAVEPOINT creates a marker within a transaction to which you can later roll back, without affecting the entire transaction. Syntax:

SAVEPOINT savepoint_name;

5.2.2 COMMIT

COMMIT permanently saves all changes made during the current transaction. Once committed, changes cannot be undone with ROLLBACK. Syntax:

COMMIT;

5.2.3 ROLLBACK

ROLLBACK undoes all changes made during the current transaction (or to a SAVEPOINT) and restores the database to its previous consistent state. Syntax:

ROLLBACK;

ROLLBACK TO savepoint_name;

5.3 Transactions for University ERP System

The following five transactions demonstrate the use of SAVEPOINT, COMMIT, and ROLLBACK in the context of the University ERP System.

5.3.1 Transaction 1: Enroll a New Student and Assign to a Subject

This transaction inserts a new student record and enrolls the student into a subject. If any step fails, the entire transaction is rolled back.

```
-- Transaction 1: New Student Enrollment
START TRANSACTION;

-- Step 1: Insert new student
INSERT INTO Student VALUES
  (104, 'Sneha', 'sneha@gmail.com', '9556677889', 1, 1);

-- Step 2: Set savepoint after student insertion
SAVEPOINT after_student_insert;

-- Step 3: Enroll student in a subject
INSERT INTO Enrollment VALUES
  (505, 104, 401, '2026', 'Active');

-- Step 4: Verify the insertion
SELECT s.Name, e.SubjectID, e.Status
  FROM Student s JOIN Enrollment e
  ON s.StudentID = e.StudentID
  WHERE s.StudentID = 104;

-- Step 5: Commit the transaction
COMMIT;
```

Output:

Name	SubjectID	Status
Sneha	401	Active

Table 5.3: Output of Transaction

5.3.2 Transaction 2: Insert Exam Result with Rollback on Error

This transaction inserts a result for a student. If the marks violate the CHECK constraint (marks > 100), the transaction is rolled back to the savepoint, preventing invalid data entry.

```
-- Transaction 2: Insert Exam Result with Rollback
START TRANSACTION;

-- Step 1: Insert valid result for Sneha
INSERT INTO Exam VALUES (704, '2026-04-10', 100, 401);

SAVEPOINT after_exam_insert;

-- Step 2: Insert valid result
INSERT INTO Result VALUES (807, 78, 'B', 7.8, 104, 704);

SAVEPOINT after_valid_result;

-- Step 3: Attempt invalid result (marks = 115, violates CHECK constraint)
-- This will fail: INSERT INTO Result VALUES (808, 115, 'A+', 9.5, 104, 703);

-- Step 4: Rollback to savepoint before the bad insert
ROLLBACK TO after_valid_result;

-- Step 5: Verify only valid result exists
SELECT * FROM Result WHERE StudentID = 104;

-- Step 6: Commit the valid changes
COMMIT;
```

Output:

ResultID	Marks	Grade	GPA	StudentID	ExamID
807	78	B	7.8	104	704

Table 5.4: Output of Transaction 2

5.3.3 Transaction 3: Update Enrollment Status with Savepoints

This transaction updates the enrollment status of multiple students. If an update on one student fails, only that change is rolled back while the rest are committed.

```
-- Transaction 3: Batch Update Enrollment Status
START TRANSACTION;

-- Step 1: Deactivate enrollment for student 101 in subject 401
UPDATE Enrollment SET Status = 'Completed'
  WHERE StudentID = 101 AND SubjectID = 401;

SAVEPOINT after_first_update;

-- Step 2: Deactivate enrollment for student 102
UPDATE Enrollment SET Status = 'Completed'
  WHERE StudentID = 102 AND SubjectID = 402;

SAVEPOINT after_second_update;

-- Step 3: Attempt update for a non-existent enrollment (no rows affected – safe)
UPDATE Enrollment SET Status = 'Completed'
  WHERE StudentID = 999 AND SubjectID = 401;

-- Step 4: Rollback only the last (bad) update
ROLLBACK TO after_second_update;
```

```
-- Step 5: Verify the first two updates are intact
SELECT StudentID, SubjectID, Status FROM Enrollment
  WHERE StudentID IN (101, 102);

COMMIT;
```

Output:

StudentID	SubjectID	Status
101	401	Completed
102	402	Completed

Table 5.5: Output of Transaction 3

5.3.4 Transaction 4: Add New Course and Subject Atomically

This transaction adds a new course and its corresponding subjects together. If inserting any subject fails, the entire transaction is rolled back to maintain consistency — a course should not exist without its subjects.

```
-- Transaction 4: Add New Course and Subjects
START TRANSACTION;

-- Step 1: Insert new course
INSERT INTO Course VALUES (303, 'BTech IT', 4, 1);

SAVEPOINT after_course_insert;

-- Step 2: Insert subjects for the new course
INSERT INTO Subject VALUES
  (404, 'Computer Networks', 4, 3, 303);

SAVEPOINT after_first_subject;
```

```
INSERT INTO Subject VALUES
  (405, 'Software Engineering', 3, 4, 303);

SAVEPOINT after_second_subject;

-- Step 3: Verify
SELECT c.CourseName, s.SubjectName, s.Credits
  FROM Course c JOIN Subject s ON c.CourseID = s.CourseID
  WHERE c.CourseID = 303;

-- Step 4: All successful, commit
COMMIT;
```

Output:

CourseName	SubjectName	Credits
BTech IT	Computer Networks	4
BTech IT	Software Engineering	3

Table 5.6: Output of Transaction 4

5.3.5 Transaction 5: Full Rollback on Constraint Violation

This transaction demonstrates a full rollback scenario where a foreign key violation causes the entire transaction to be undone, ensuring no partial data is saved.

```
-- Transaction 5: Rollback on Constraint Violation
START TRANSACTION;

-- Step 1: Insert a valid faculty member
INSERT INTO Faculty VALUES (204, 'Dr Nair', 'Cloud Computing', 1);
```

```
SAVEPOINT after_faculty_insert;

-- Step 2: Assign timetable for the new faculty
INSERT INTO Timetable VALUES
  (604, 'Thursday', '12:00-13:00', 'C301', 204, 401);

SAVEPOINT after_timetable_insert;

-- Step 3: Attempt to insert result with invalid ExamID (FK violation)
-- This will fail: ExamID 999 does not exist
-- INSERT INTO Result VALUES (809, 82, 'A', 8.2, 101, 999);

-- Step 4: Since the above violates FK, rollback everything
ROLLBACK;

-- Step 5: Verify Faculty 204 was NOT inserted (full rollback)
SELECT * FROM Faculty WHERE FacultyID = 204;
```

Output:

Operation	Result
SELECT Faculty WHERE FacultyID = 204	Empty Set (0 rows) — Full ROLLBACK successful

Table 5.7: Output of Transaction 5 – Full Rollback Verified

5.4 Concurrency Control

Concurrency control ensures that multiple transactions can execute simultaneously without causing data inconsistency or integrity violations. In the University ERP System, concurrency control is crucial when multiple users (e.g., faculty entering results, students enrolling) access the database at the same time.

5.4.1 Concurrency Control Algorithms

MySQL implements concurrency control primarily through locking mechanisms. The main approaches are:

Algorithm	Description
Two-Phase Locking (2PL)	Transactions acquire all locks before releasing any. Growing phase: locks acquired. Shrinking phase: locks released. Guarantees serializability.
Timestamp Ordering	Each transaction is assigned a timestamp. Operations are ordered by these timestamps to ensure consistency.
MVCC (Multi-Version Concurrency Control)	MySQL InnoDB uses MVCC. Readers do not block writers; each transaction sees a snapshot of data consistent with its start time.

Table 5.8: Concurrency Control Algorithms

5.4.2 Locking Commands in MySQL

a. Row-Level Locking – SELECT ... FOR UPDATE

Locks specific rows for the duration of the transaction. Other transactions are blocked from modifying the locked rows until the current transaction commits or rolls back.

```
-- Lock a student's result row for exclusive update
START TRANSACTION;
SELECT * FROM Result
  WHERE StudentID = 101 FOR UPDATE;
-- Perform updates on the locked row
UPDATE Result SET Marks = 90, Grade = 'A+'
  WHERE StudentID = 101 AND ExamID = 701;
COMMIT;
```

b. Table-Level Locking – LOCK TABLES

Locks an entire table to prevent other sessions from reading or writing during critical operations such as bulk updates.

```
-- Acquire a write lock on the Enrollment table
```

```
LOCK TABLES Enrollment WRITE;

-- Perform bulk update
UPDATE Enrollment SET Status = 'Archived'
  WHERE AcademicYear = '2024';

-- Release the lock
UNLOCK TABLES;
```

Lock Modes Reference:

Lock Mode	Description
ROW SHARE	Allows concurrent read access; prevents exclusive table locks.
ROW EXCLUSIVE	Used for DML operations; prevents other sessions from locking in share mode.
SHARE	Allows queries but blocks updates and deletes from other sessions.
SHARE ROW EXCLUSIVE	More restrictive than SHARE; prevents concurrent updates.
EXCLUSIVE	Full table lock. Prevents all other access including reads.

Table 5.9: MySQL Lock Modes

c. COMMIT – Release All Locks

```
COMMIT;  -- Commits all changes and releases all row/table locks.
```

d. ROLLBACK – Undo Changes and Release Locks

```
ROLLBACK;  -- Undoes all changes since the last COMMIT and releases all locks.
```

5.4.3 Concurrency Control Example for University ERP System

Scenario: Two faculty members (Session A and Session B) are simultaneously trying to update the result of Student 101 for Exam 701. Locking ensures that only one session modifies the record at a time.

Session A (Faculty 1)	Session B (Faculty 2)
START TRANSACTION; SELECT * FROM Result WHERE StudentID = 101 FOR UPDATE; -- Row locked by Session A	START TRANSACTION; SELECT * FROM Result WHERE StudentID = 101 FOR UPDATE; -- WAITING... blocked by Session A

```
-- Session A continues:
UPDATE Result SET Marks = 92, Grade = 'A+'
  WHERE StudentID = 101 AND ExamID = 701;
COMMIT;  -- Lock released

-- Session B now proceeds (lock acquired after Session A commits):
-- Session B sees updated value (Marks = 92) and can now update
UPDATE Result SET GPA = 9.2
  WHERE StudentID = 101 AND ExamID = 701;
COMMIT;
```

Output:

ResultID	Marks	Grade	GPA
801	92	A+	9.2

Table 5.10: Output after Concurrent Update with Locking

Result: Row-level locking ensured that Session A completed its update before Session B could modify the same record. This prevents lost updates and maintains data consistency, demonstrating effective concurrency control in the University ERP System.

CHAPTER 6

FRONT-END AND BACK-END CODE OF <PROJECT TITLE>

6.1 Front–End Module codes

6.2 Database connectivity

CHAPTER 7

RESULTS AND DISCUSSIONS

7.1 Screenshots of front-end

7.2 Screenshots of Database

References